

Practical 3: E-R modelling

Objectives

After completing this experiment, you will be able to:

- Identify entity sets, their attributes, and various relationships
- Represent the data model through ER diagram

Entity Relationship Model:

Entity-Relationship model is used to represent a logical design of a database to be created. In ER model, real world objects (or concepts) are abstracted as entities, and different possible associations among them are modelled as relationships.

For example, student and school -- they are two entities. Students study in school. So, these two entities are associated with a relationship "Studies in".

As another example, consider a system where some job runs every night, which updates the database. Here, job and database could be two entities. They are associated with the relationship "Updates".

Importance of ER Modelling:

Figure - 01 shows the different steps involved in implementation of a (relational) database.



Figure - 01: Steps to implement a RDBMS

Given a problem statement, the first step is to identify the entities, attributes and relationships. We represent them using an ER diagram. Using this ER diagram, table structures are created, along with required constraints. Finally, these tables are normalized in order to remove redundancy and maintain data integrity. Thus, to have data stored efficiently, the ER diagram is to be drawn as much detailed and accurate as possible.

Case Study: Cloud Gaming Console Platform for GameSphere Technologies

A robust database backend is essential for developing a reliable and efficient Cloud Gaming Console Platform. The database schema should be carefully designed, refined, and normalized to ensure efficient management of users, games, cloud servers, and administrative operations. In this section, we develop a simple Entity Relationship (ER) model for the proposed Cloud Gaming Console Platform.

The first step towards ER modelling is to identify the set of relevant entities from the given problem statement. The primary and most important entity sets in this context are "User", "Game", "Server", and "Admin".

The "User" entity set represents all registered players who create an account on the platform to access cloud gaming services. Each user has a unique User ID along with personal information such as username, email, password, subscription details, and account status.

→ Figure 1 represents the "User" entity set and its primary key.

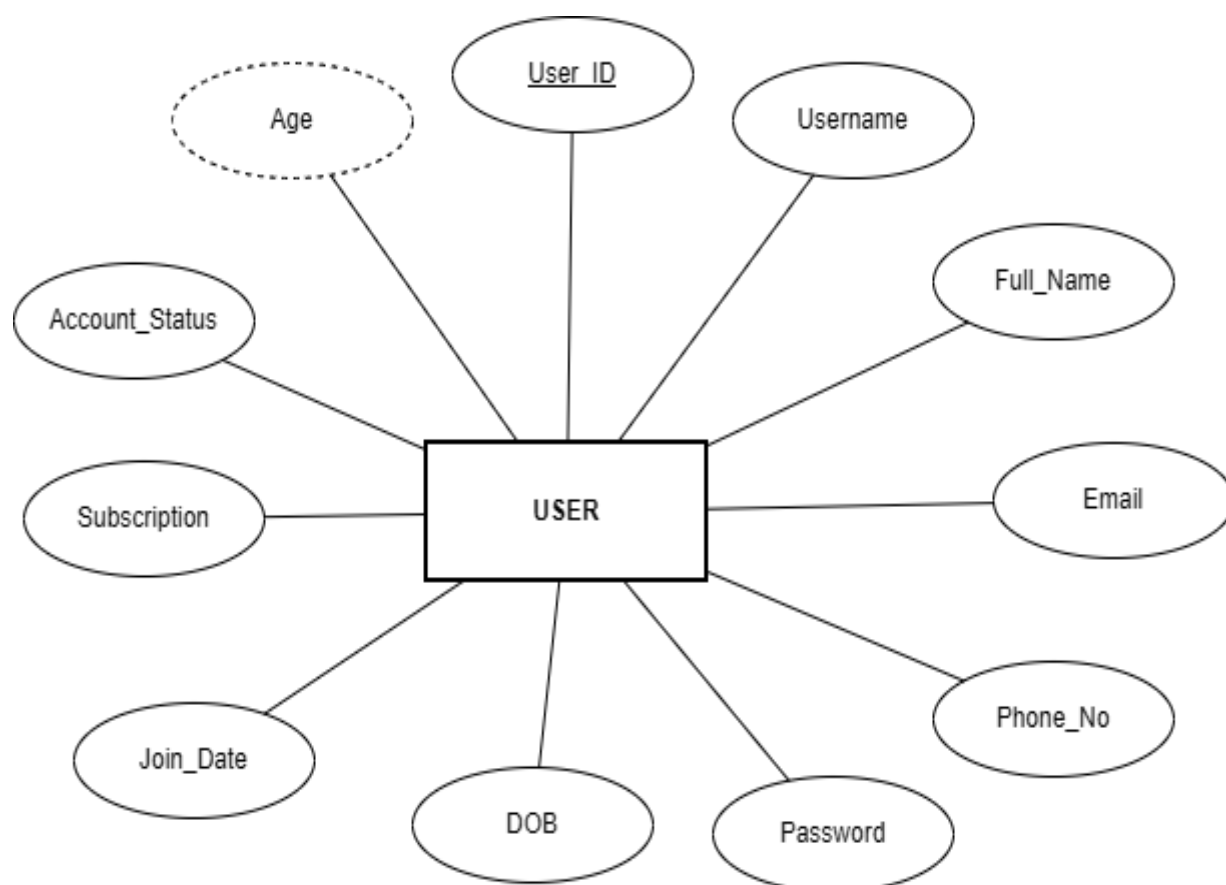


Figure 1: "User" Entity Set

Let us now focus on the "Game" entity set. This entity contains information about every game available on the cloud gaming platform. Each game is identified by a unique Game ID and stores attributes such as game name, genre, developer, release date, rating, and subscription requirement.

→ Figure 2 represents the "Game" entity set.

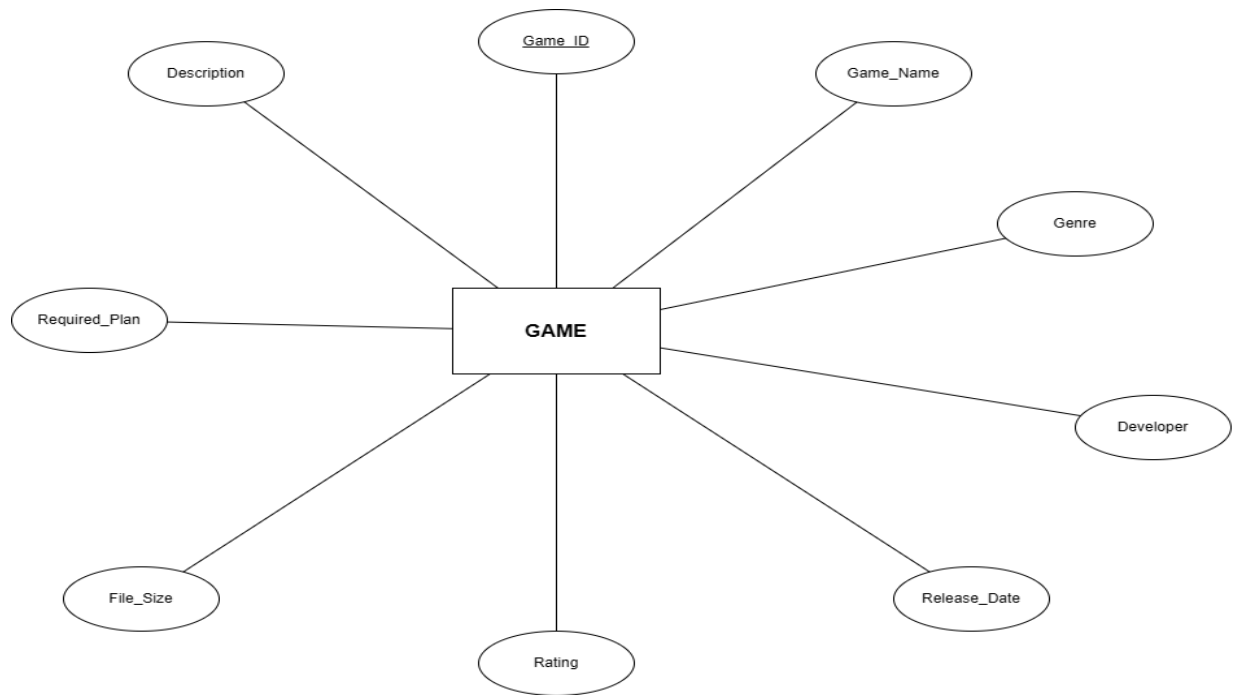
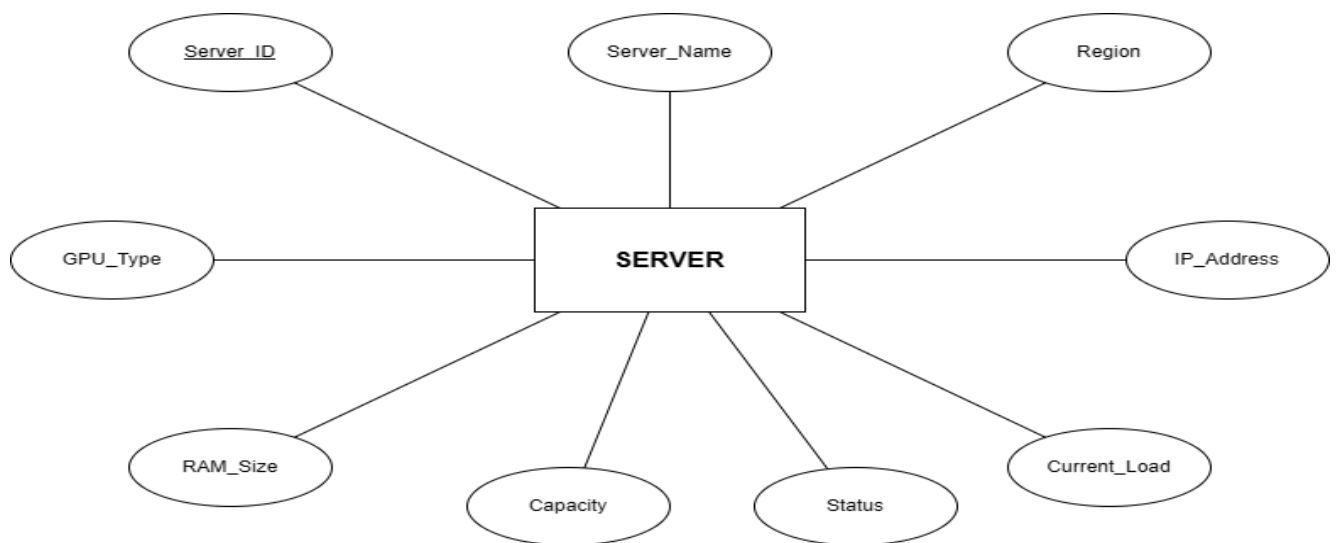


Figure 2: "Game" Entity Set

The "Server" entity set stores information about cloud servers responsible for hosting and streaming games. Each server has a unique Server ID along with attributes such as server name, region, IP address, GPU type, RAM size, capacity, current load, and operational status.

→ Figure 3 represents the "Server" entity set.



→ Figure 3: "Server" Entity Set

The "Admin" entity set represents administrators responsible for managing the Cloud Gaming Console Platform. Administrators maintain user accounts, manage games, monitor cloud servers, and supervise the overall functioning of the system.

Each administrator is identified by a unique Admin ID.

→ Figure 4 represents the "Admin" entity set.

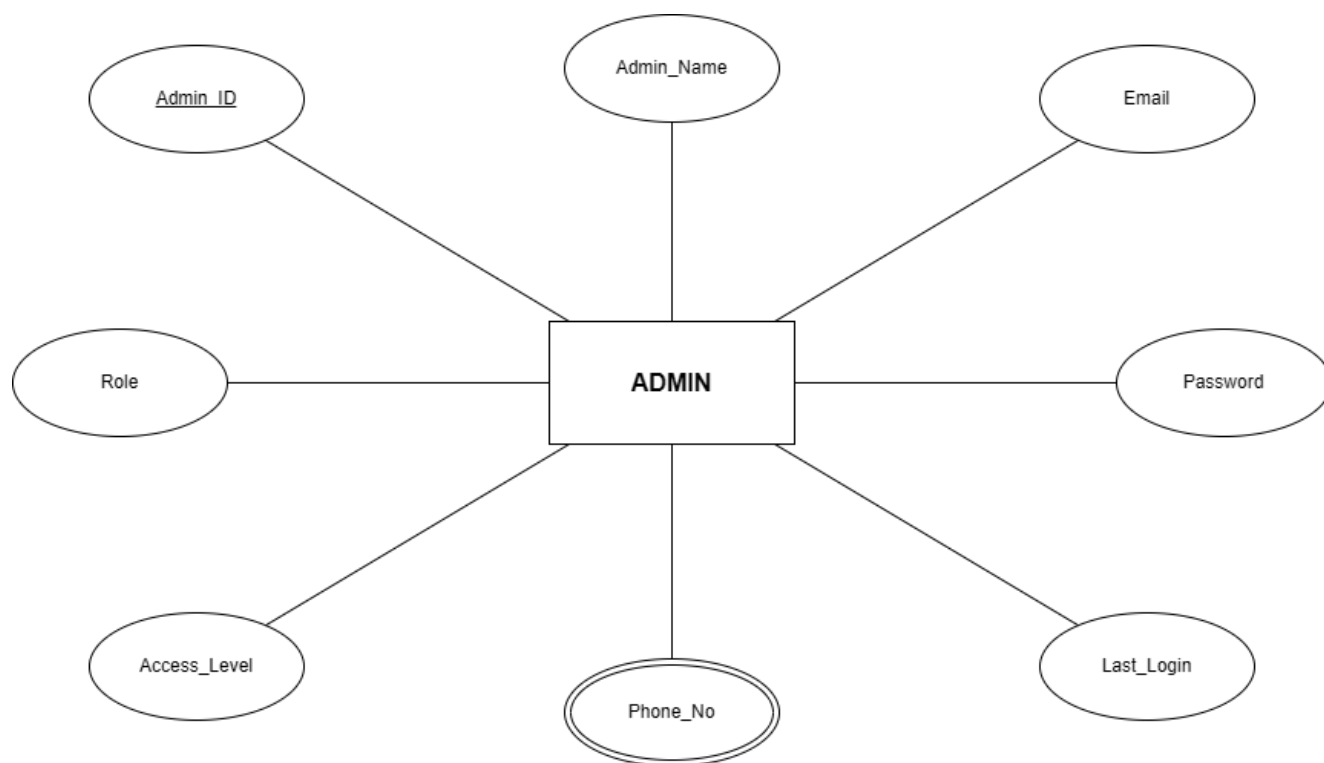


Figure 4: "Admin" Entity Set

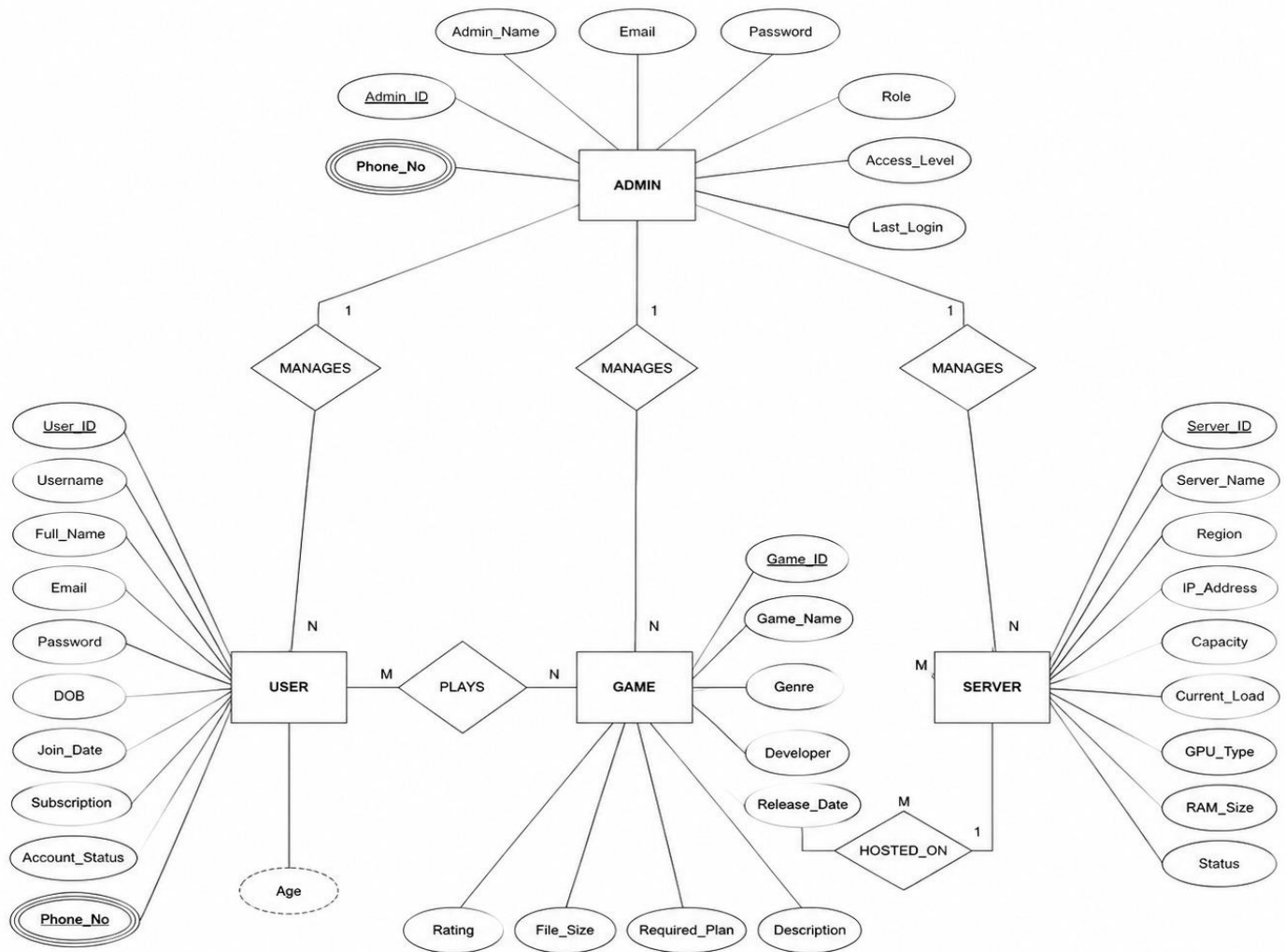
The relationship among the identified entity sets is illustrated in Figure 5.

A User can play multiple Games, and each Game can be played by many different users. Therefore, the relationship between User and Game is a many-to-many (M:N) relationship.

Similarly, every Game is hosted on a Server. A single server can host multiple games depending on its available resources, whereas each game is assigned to one active server during deployment. Hence, the relationship between Game and Server is many-to-one (M:1).

The Admin manages multiple Users, Games, and Servers. One administrator can manage many users, add or update multiple games, and monitor several cloud servers simultaneously. Therefore, these relationships are one-to-many (1:N).

→ Figure 5: Relationships among Different Entity Sets.



The ER diagram also shows that the Admin is responsible for managing the cloud gaming infrastructure. The administrator can add new games to the platform, remove outdated games, update game information, allocate cloud servers, monitor server health, and manage registered user accounts.

The relationship between Admin and Game, User, and Server is one-to-many because a single administrator can manage multiple records of each entity, while every managed entity is supervised by one administrator at a given time.